

Weekly Report Content

General Information

- Group ID: 53
- Group name: Echoes of the Ashen Vault.
- Project name: Echoes of the Ashen Vault (Dungeon Game).
- Date range: 06.07.2026 - 11.07.2026

Tasks completed this week

25125027 - Nguyễn Tấn Minh:

- Design and source/create sprite assets: player character (3 forms × idle/walk/attack frames), basic enemy sprites (WaterloggedScribe, ShardSoldier, BoneSprinter), tilemap tiles (stone floor, water, walls, barriers).
- Implement `Stats` and `StatModifier` structs. Add flat `defense` stat to forms (Wraithblade: 15, Voidcaster: 5, Ironshell: 35).
- Implement the Defense mitigation formula in `Character::calculateMitigatedDamage(rawAmount)`: `mitigated = rawAmount * 100 / (100 + defense)`, rounded, clamped to min 1. Wire into `Character::takeDamage(rawAmount)` as the single entry point for all hits.
- Implement decoupled visual Observer pattern: `CharacterObserver` interface (`onStateChanged`, `onDamaged`, `onDefeated`) and `CharacterAnimator` class (managing sprite, current animation key, hit flash timer, observers callbacks). Integrate with `Character::draw()`.
- Implement `SpriteSheet` class: load texture atlas, define frame rects, support multi-row layouts for different animation states.
- Implement `Animation` class: frame sequencing, timing (`frameDuration`), looping/non-looping modes, `play()`, `pause()`, `reset()`, `getCurrentFrame()`.
- Refactor `Player` to use `PlayableCharacter` abstract factory (per architecture.puml): `Player` takes a `PlayableCharacter&` in constructor, stores forms in `std::map<FormType, std::unique_ptr<PlayerForm>>`, generalizes momentum to `std::map<FormType, float>`. Implement `Serin` as the concrete `PlayableCharacter` whose `createForm1/2/3()` return `WraithbladeForm`, `VoidcasterForm`, `IronshellForm`. Decouple `Player` from specific forms.
- Implement `PlayerCombatState` interface and `PlayerCombatStateMachine` (State pattern) to manage active combat states. Formulate `PlayerForm` as a concrete subclass of `PlayerForm` implementing `PlayerCombatState`.
- Implement `StatusEffect` abstract base class with `apply(Character&)`, `remove(Character&)`, `update(dt, Character&) → bool` (per architecture.puml).
- Implement `BurnedEffect`: ticks once per second for 10 seconds, dealing `0.25 × dealerBaseDamage` per tick (affected by target defense).
- Implement `ParalyzedEffect`: 10-second duration; exposes `rollMiss()` returning true 40% of the time, re-rolled per action attempt.
- Implement `SlowedEffect`: sets effective move speed to `base_speed × 0.70` via `StatModifier`; supports a `permanent` flag for zone-based slow that ignores the 10-second expiry rule.
- Wire `Character::applyStatusEffect(unique_ptr<StatusEffect>)` and `Character::tickStatusEffects(float dt)` into `Character::update()`.

- Implement player melee and ranged attack hitboxes that scale by active form parameters.

25125061 - Châu Tấn Phát:

- Implement `Enemy` abstract class inheriting from `Character`. Add `playerRef : Player&`, `aiState : EnemyAIState*`, `attackCooldown`, and `updateAI(dt, Chamber&)` method.
- Implement the **State Pattern** for enemy AI: `EnemyAIState` interface and concrete classes (`IdleAIState`, `ChasingAIState`, `AttackingAIState`, `StaggeredAIState` with timed auto-revert mechanism).
- Add `isCarrier`, `isDecoy`, `fragmentDropCount`, `isRealCarrier()` virtual method, and `onDeath()` abstract method to `Enemy`.
- Implement `EnemyFactory::createEnemy(EnemyType, Player&) → unique_ptr<Enemy>` (per `architecture.puml`).
- Code simple steering AI behaviors: **Seek** (chases player position) and **Evade** (moves away from threat) as shared methods on `Enemy`.
- Implement `WaterloggedScribe` (HP 18, Dmg 5, Speed 1.5 — effective after chamber water-slow; no additional Slowed multiplier unless additionally afflicted).
- Implement `ShardSoldier` (HP 16, Dmg 6). Include a `selfHealActive` flag and `applySelfHeal(float dt)` method.
- Implement `BoneSprinter` (HP 14 carrier / HP 20 blocker, Speed 9.0 / 6.0). Include `isCarrier` flag for carrier/blocker variants.
- Add `BONE_SPRINTER` and `SHARD_WRAITH` to `EnemyType` enum in `enums.hpp`.
- Integrate enemy sprites from Dev A's sprite assets.

AI Usage Declaration

- **Claude**. Anthropic, accessed 11/07/2026. Purpose: Generate an alias file for LPC-format spritesheets.
- Other tasks done by AI agents using Gemini model, via Antigravity:
 - Timer-based status effect tick boilerplate.
 - Standard steering behavior algorithms (Seek, Flee, Obstacle Avoidance).
 - Class constructors for basic enemies via the factory.
 - SpriteSheet frame-rect calculation utilities.

Issues

No issues encountered during this week.